

复习

什么是算法 — 定义

算法是解决问题的方法或过程，严格地讲是满足下述性质的指令序列。

- ◆ 输入：有零个或多个外部量作为算法的输入。
- ◆ 输出：算法产生至少一个量作为输出。
- ◆ 确定性：组成算法的每条指令清晰、无歧义。
- ◆ 有限性：算法中每条指令的执行次数有限，执行每条指令的时间也有限。

什么是算法 — 计数

- 判断一个byte里面有多少个bit的值为1?

```
int Cal(int iValue){
    while(iValue!=0){
        iReminder=iValue%2;
        if(iReminder==1)
            iCount++;
        iValue=iValue/2;
    }
    return iCount;
}
```

- 如果这个子函数需要调用很多次，内存空间足够，怎样提高性能？
- 如果既想省时间，又想省空间，怎样改进？

```
int iTables[256] = {...};
iCount =
iTables[iValue];
```

```
HashMap Maps;
If(Maps. containsKey (Value)){
    Count=Maps.get(Value)}
else{
    Maps.put(Value,new Integer(Cal(iValue)))}
```

时间复杂度分析

- ◆ 时间复杂性函数 $T(N, I)$ 的定义
- ◆ 渐进复杂性函数的定义，知道怎样按照渐进阶进行排序
- ◆ 记号 $\mathcal{O}$ 的定义和使用

课程内容

- 算法复杂性
- 递归与分治
- 动态规划
- 贪心算法
- 回溯法
- 分支限界法

分治策略

分治法所能解决的问题特征：

- ◆ 该问题的规模缩小到一定的程度就可以容易地解决
- ◆ 该问题可以分解为若干个规模较小的**相同**问题
- ◆ 利用该问题分解出的子问题的解可以**合并**为该问题的解
- ◆ 该问题所分解出的各个子问题是相互独立的

治众如治寡，分数是也！

```
divide-and-conquer(P)
{
  if ( | P | <= n0) adhoc(P);           //解决小规模的问题
  divide P into smaller subinstances P1,P2,...,Pk; //分解问题
  for (i = 1; i <= k; i++)
    yi = divide-and-conquer(Pi); //递归的解各子问题
  return merge(y1,...,yk);           //将各子问题的解合并为原问题的解
}
```

合并排序、快速排序、二分查找

下列关于分治算法的描述，错误的是（）

- ☐ A 二分查找、归并排序、快速排序均属于分治思想的典型应用
- ☒ B 分治算法的子问题结构与原问题完全不同
- ☐ C 分治要求子问题之间相互独立，不存在重叠子问题
- ☐ D 分治算法遵循分解、求解、合并三步流程

提交

快速排序是应用最广泛的分治排序算法，它属于不稳定排序，其根本原因是（）

- ☐ A 递归调用产生额外开销
- ☒ B 分区过程中存在元素跳跃式交换，打乱等值元素相对位置
- ☐ C 最坏时间复杂度为 $O(n^2)$
- ☐ D 排序需要额外辅助空间

提交

使用分治法求解数组最大值和最小值，其时间复杂度为（）

A

$O(\log n)$

B

$O(n \log n)$

C

$O(n)$

D

$O(n^2)$

提交

动态规划

前事不忘，后事之师

1. 分析最优解的性质，并刻画其**最优子结构**特征
2. 确定**状态表示** $S(x_1, x_2, \dots)$ 和**状态转移方程**，递归地定义最优值
3. 确定**状态转移顺序**，以自底向上的方式计算出最优值
4. 根据计算最优值时得到的信息，**构造最优解**

矩阵连乘、多段图最短路径、单源最短路径、0-1背包问题、最长公共子序列

- 设有四个矩阵 $M1$ 、 $M2$ 、 $M3$ 、 $M4$ ，它们的维度分别为 3×5 、 5×2 、 2×7 、 7×4 。请运用动态规划算法求解矩阵连乘 $M1 \times M2 \times M3 \times M4$ 的最少乘法运算次数。

贪心算法也是一种**基于子问题思想**的策略。贪心算法是一个分阶段决策过程，在每个局部阶段，贪心法都做出一个**当前最优**的局部决策，并期望通过每次所做的**局部最优决策**产生一个**全局最优解**。

贪心算法的三个逻辑步骤：

- **分解** 将原问题求解过程划分为**连续的若干个决策阶段**
- **决策** 在每一个阶段依据贪心策略进行**局部贪心决策**，并**缩小**待求解问题的规模
- **合并** 将各个阶段的**局部解合并**为原问题的一个全局可行解

活动安排问题、哈夫曼编码、分数背包问题、最小生成树

已知无向图 G 的邻接矩阵 $A = \begin{bmatrix} \infty & 1 & \infty & 6 & 3 \\ 1 & \infty & 8 & \infty & 7 \\ \infty & 8 & \infty & 4 & 5 \\ 6 & \infty & 4 & \infty & 2 \\ 3 & 7 & 5 & 2 & \infty \end{bmatrix}$ 。

- (a) 画出邻接矩阵 A 所表示的无向图 G ;
- (b) 给出所有从顶点1出发的广度优先遍历序列;
- (c) 根据Kruskal/Prim算法画出最小生成树, 需要给出每一步的生成过程。

有4个字符a~d，它们的频度分别为8，1，4，6。

问题：

- (1) 构造其哈夫曼编码树，并求平均码长是多少？
- (2) 说明将“aabcabd”字符串编码结果；
- (3) 请给出“10010011000”的解码结果。

算法篇 - 搜索策略

搜索算法是一种**通用的问题求解方法**：首先把问题表示转换为一个状态空间图，然后设计特定的**图遍历方法**在状态空间中搜索问题的解

搜索算法的两个逻辑步骤：

1. 定义问题的**解空间**
2. **搜索**解空间，并在搜索过程中用**剪枝策略**避免无效搜索

■ 基于枚举策略的搜索

- 深度优先搜索(DFS)
- 广度优先搜索(BFS)

优化枚举，提前判定

■ 优化+枚举的搜索

- 回溯算法 = 深度优先搜索 + 剪枝策略
- 分支限界算法 = 广度优先搜索 + 剪枝策略

0-1背包问题、n皇后问题、装载问题、单源最短路径问题

算法人生诗

分治如刀破万难，递归拆解悟真章。

动规积步行千里，记忆存解岁月长。

贪心拾贝迷歧路，短视难窥全局光。

回溯可退重择径，频顾反困旧时光。

算法藏机代码里，人生如程悟短长。

考试信息

时间： 06月30日（周二） 13:30-15:20

地点： 教2—409

形式： 闭卷

题型： 填空题、判断题、选择题、简答题、算法设计与分析题

谢谢 THANKS